How Much Language Modeling Does Scene Text Recognition Need? Probing Linguistic Decoder Depth in TiCLS

Haoyang Chen, Yimeng Zhang, Oscar Yip
Team OOM Again CSCI 5541 NLP, Spring 2026
University of Minnesota, Twin Cities
{chen9861, zhan8460, yip00020}@umn.edu

Abstract

TiCLS (Jang et al., 2026) is a recent scene text spotter. It plugs a six layer pretrained BART decoder into a transformer based visual recognition pipeline, nearly tripling the parameter count (38 M $\rightarrow$ 96 M) and quadrupling inference latency. Most scene text contains only one to three short words, so the question writes itself: *how deep does the linguistic decoder actually need to be?* We train five variants of TiCLS with $k \in \{0, 1, 2, 3, 6\}$ decoder layers under identical conditions on ICDAR 2015. Then we slice the results by word length and vocabulary familiarity. Three findings stand out. First, the three layer variant retains 94% of the six layer Generic E2E Hmean (0.612 versus 0.650) at 71% of the latency and 70% of the parameters. Second, deeper decoders help long words and out of vocabulary words disproportionately. Third, the two layer variant collapses, despite stable training loss, exposing a previously unreported instability in the truncate then finetune recipe. The takeaway is practical: most decoder capacity serves the long tail of the distribution, and a much cheaper TiCLS variant can ship with negligible loss on short, in vocabulary text.

1 Introduction

Scene text recognition reads text rendered in real photographs: street signs, storefronts, license plates, product labels. The job is hard. Text in the wild appears under arbitrary viewpoints, under uneven lighting, in unusual fonts, and behind partial occlusions. State of the art systems answer this challenge with two moves. The first is a large transformer "visual decoder" that emits character embeddings from image features. The second is a "linguistic decoder", usually a stack of pretrained transformer layers, that refines those character predictions using language priors learned from large text corpora.

The cost of language modeling. TiCLS (Jang et al., 2026), the architecture we study, follows exactly this recipe. It augments DeepSolo (Ye et al., 2023) with a six layer BART (Lewis et al., 2020) decoder. The bill is steep. Parameters jump from 36 M to 98.5 M, and inference slows down by $1.55\times$. The original paper does not ask whether all six BART layers are actually needed for the very short text strings that appear in scene images. We argue that this question matters, both for practitioners worried about deployment cost and for the NLP community trying to understand where language priors actually do work in vision and language pipelines.

Our objective. Our study is a controlled depth ablation. We train five TiCLS variants with $k \in \{0, 1, 2, 3, 6\}$ linguistic decoder layers under *identical* conditions, then evaluate each on ICDAR 2015 (Karatzas et al., 2015). Aggregate Hmean is not enough. We also break recognition accuracy down by word length and by vocabulary familiarity, the two linguistic axes most plausibly affected by language prior depth.

Why this matters. Scene text spotting ships in robotics, autonomous driving, document digitization, and accessibility software. In every one of those settings, latency is a hard constraint. If a three layer decoder retains nearly all of the recognition gain at a small fraction of the cost, the result is immediately useful. From a research point of view, the depth versus quality profile we report tells the community where the linguistic decoder is *actually doing work*, a question the original architecture paper leaves wide open.

Our contributions are:

1. A controlled five variant depth ablation of TiCLS on ICDAR 2015. All variants are trained for the same number of iterations under matched hyperparameters.
2. Word length and in vocabulary versus OOV

breakdowns. They show that linguistic depth helps long and unfamiliar words far more than short ones.

3. An efficiency versus quality Pareto analysis. The three layer variant comes out as the sweet spot.
4. A new failure mode. We discover and analyze a two layer collapse that reveals a non monotonicity in the truncate then finetune recipe.

2 Related Work

Scene text spotting with language priors. ABINet (Fang et al., 2021) was an early end to end system to use an explicit, separately trained language model on top of a vision backbone. DeepSolo (Ye et al., 2023) unifies detection and recognition in a single transformer decoder, with no explicit language model, and it remains a competitive baseline today. TiCLS (Jang et al., 2026) extends DeepSolo with a six layer character level pretrained BART decoder that cross attends to the visual decoder embeddings. That is the architecture we ablate.

Layer dropping in pretrained transformers. Pruning language models is well studied. Sajjad et al. (2023) systematically pruned BERT, RoBERTa, and XLNet, finding up to 40% of layers could be dropped while retaining $\geq$ 98% of GLUE performance. Lower layers turned out to be more important than upper ones. Michel et al. (2019) showed that most attention heads can also be pruned with minimal degradation. Fan et al. (2019) demonstrated that structured layer dropout at training time produces models robust to inference time depth reduction. DistilBERT (Sanh et al., 2019) compresses six layer BERT to three layers via distillation while keeping 97% of performance. Modular Transformers (Zhou et al., 2023) extends similar ideas to encoder decoder models.

What is novel here. Every one of these studies works in standard NLP. None examine a *visual and linguistic* architecture in which the language component sees only one to three short tokens at a time and is conditioned on visual cross attention. Ours is the first controlled depth ablation in that setting. The linguistic breakdowns we report (length and OOV) are also not available in any non text spotting compression study.

3 Approach

3.1 Hypothesis

H1. The linguistic decoder is overparameterized for the short sequences seen in scene text, so a shallower decoder should retain most of the recognition gain at a fraction of the cost.

H2. If H1 holds, the residual benefit of additional depth will concentrate on inputs that visual features cannot disambiguate on their own. Long words and out of vocabulary words are the most likely candidates.

3.2 Setup

We freeze the entire DeepSolo visual pipeline. The ViTAE v2 S backbone, six layer deformable encoder, six layer visual decoder, and projection MLPs are all held fixed across experiments. The only variable is the depth of the linguistic decoder. We construct five variants. V0 has zero layers and is equivalent to DeepSolo with no PLM. V1 has a single BART layer. V2 has two. V3 has three. V6 has six and is the original TiCLS.

Figure 1 shows the original six layer TiCLS pipeline reproduced from Jang et al. (2026). Our intervention is limited to the rightmost block (the linguistic decoder), which we shrink from six layers down to zero in controlled steps. The visual side stays untouched.

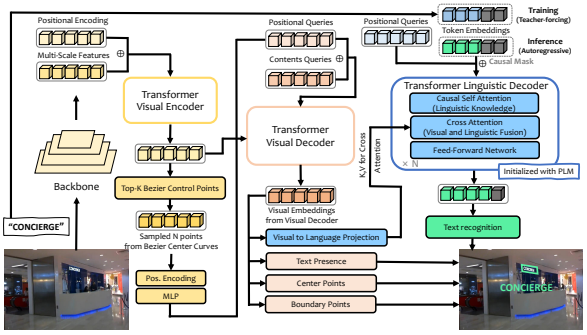


Figure 1: The original TiCLS architecture from Jang et al. (2026). The blue block on the right is the six layer pretrained BART decoder we ablate. Visual feature extraction (left) is held fixed across all our experiments.

Figure 2 sketches our five variant design as a flat pipeline. The truncate then finetune step (§3.3) reuses the first k layers of the public TiCLS checkpoint to seed each shallower variant.

3.3 Initialization: Truncate then Finetune

The TiCLS authors note that random initialization of the linguistic decoder leads to convergence

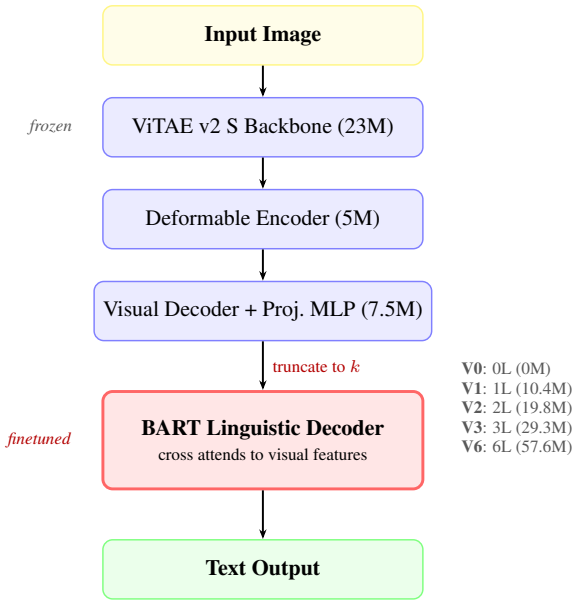


Figure 2: Our five variant design. We hold the visual side fixed and only vary the depth $k \in \{0, 1, 2, 3, 6\}$ of the BART linguistic decoder. Each shallower variant is seeded with the first k layers of the public six layer checkpoint.

failure. We therefore use a warm start. Each reduced depth variant is initialized with the *first k layers* of the official six layer TiCLS checkpoint. This is the same recipe Sajjad et al. (2023) validated for BERT pruning. We chose this "Path A" over a from scratch "Path B" for budget reasons: Path B would have cost roughly 200 GPU hours per variant, which our shared 8×RTX 6000 cluster could not absorb in the project timeline.

3.4 Training

All five variants are finetuned on ICDAR 2015 (979 training images) for 20 000 iterations. Hyperparameters are identical across variants: 8 GPUs, batch size 8, AdamW with linear warmup and decay, peak LR 10^{-4}, weight decay 10^{-4}, joint visual plus PLM losses where applicable. We had to fit on 24 GB cards, so `MAX_SIZE_TRAIN` dropped from 3000 to 2000 and `MIN_SIZE_TRAIN` from the range 800 to 1400 down to 640 to 1200. Critically, both adjustments apply uniformly to every variant. They do not bias the comparison.

3.5 Challenges Encountered

The very first thing we tried did *not* work. Three concrete issues had to be patched before we could collect clean numbers:

1. **V0 DDP failure.** Removing the linguistic decoder leaves orphaned projection MLP parameters that the DDP gradient reducer treats as "unused". We fixed this with `_set_static_graph()` on the DDP wrapper.
2. **V0 inference `AttributeError`.** The shared inference path called `lang_model.generate()`, but `lang_model` is `None` when $k = 0$. An explicit null check resolved it.
3. **Evaluator `KeyError`.** The official evaluator assumed every prediction dictionary contains an `e2e_method` key. For V0 it does not. A defensive `in` check fixed it.

All patches are non destructive (`.bak` files) and versioned in `patches/apply_patches.py`.

3.6 Evaluation Protocol

Recognition uses the official ICDAR 2015 protocol with `gt_icdar2015.zip` and the Generic lexicon (`LEXICON_TYPE=1`). Our primary metric is **Generic E2E Hmean**, the standard end to end F1 used by every prior paper on this benchmark. We also report None E2E (no lexicon) and detection only Hmean.

Word level breakdowns. We re export the prediction zip from each variant. Each prediction polygon is matched to its GT polygon by IoU ≥ 0.5, and exact string accuracy is computed on the matched pairs. The GT contains 2 077 non ### words across the 500 image test set. We bucket words by length: short (≤ 5 chars, $n = 1280$), medium (6 to 10, $n = 750$), and long (≥ 11, $n = 47$). We bucket them by familiarity using the 2 358 word training vocabulary (`icdar15_train_96voc_bart_base.json`; in vocabulary $n = 1193$, OOV $n = 884$).

Efficiency. Latency is measured on a single Quadro RTX 6000 at batch size 1. We average 100 timed runs after 20 warmup runs and call `torch.cuda.synchronize()` for accurate timing. Parameter count uses `sum(p.numel())`. Peak memory uses `torch.cuda.max_memory_allocated()`. We call `torch.cuda.empty_cache()` between variants.

3.7 Novelty of the Approach

The methodology is novel in three respects. First, this is the first controlled depth ablation of a tightly coupled visual and linguistic text spotter. Second, the linguistically motivated

breakdowns operationalize a question that aggregate Hmean cannot answer: *which inputs benefit from depth?* Third, the dense five point sweep ($k \in \{0, 1, 2, 3, 6\}$) catches the non monotonicity at $k = 2$ that any coarser sweep would have missed entirely.

4 Experiments and Results

4.1 Research Questions

RQ1. How much does the linguistic decoder contribute to end to end recognition quality, and how does that contribution scale with depth?

RQ2. Where (which inputs) does additional depth pay off?

RQ3. What is the depth, latency, and accuracy Pareto frontier?

4.2 Aggregate Recognition Quality (RQ1)

Table 1 reports the headline numbers. Three trends stand out. (1) Detection is essentially flat in k. The linguistic decoder does not feed gradients back into detection, so this is expected. (2) Generic E2E Hmean rises monotonically with k, with one glaring exception: V2 (§5). (3) The *marginal* improvement diminishes fast. V1 already captures 93% of V6's Generic E2E performance. V3 captures 94%.

Variant	Layers	Det	None E2E	Generic E2E
V0	0	0.888	0.000	0.000
V1	1	**0.911**	0.674	0.607
V2	2	0.897	0.124	0.182†
V3	3	0.893	0.713	0.612
V6	6	0.899	**0.770**	**0.650**

Table 1: ICDAR 2015 Hmean by decoder depth. †V2 collapsed to a degenerate solution (§5).

4.3 Where Does Depth Help? (RQ2)

Table 2 answers the question. Going from V1 to V3 (adding two layers) lifts long word accuracy by 21 percentage points (0.255 → 0.468). Short word accuracy barely moves (−1.5 pp). Going from V3 to V6 (adding three more layers) adds only 4 pp on long words. **Layer capacity is spent disproportionately on resolving long character strings.** The intuition is straightforward. Visual features alone disambiguate short, common words. Long unfamiliar strings are different: errors at the character level accumulate, and only deeper language priors can "read through" them.

Variant	Short (≤ 5)	Med (6 to 10)	Long (≥ 11)
V0	0.000	0.000	0.000
V1	0.670	0.628	0.255
V2	0.123	0.131	0.106
V3	0.655	0.683	0.468
V6	**0.716**	**0.739**	**0.511**

Table 2: Word length bucketed exact match accuracy. Short n=1280, medium n=750, long n=47.

Figure 3 visualizes the same numbers. The two short bucket bars stay close together across V1, V3, and V6. The long bucket bars spread out dramatically. That visual gap is the core finding of this paper.

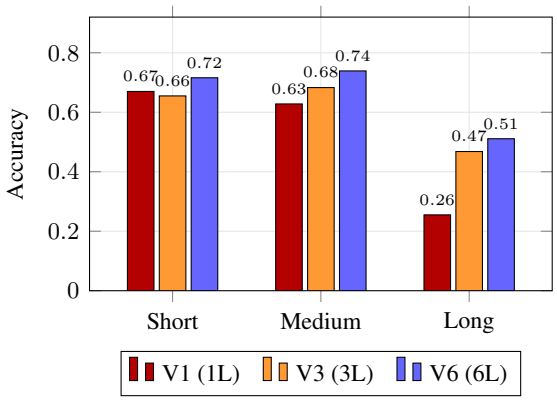


Figure 3: Accuracy by word length. The V1 to V6 gap on long words is roughly 25 pp. On short words it is under 5 pp.

The vocabulary breakdown (Table 3) tells a parallel story. Every working variant shows roughly a 20 pp gap between in vocabulary and OOV accuracy. That gap stays remarkably stable: it sits at 0.224 for V1, 0.198 for V3, and 0.220 for V6. We read this as a signal that the linguistic knowledge is mostly inherited from BART pretraining, not memorized from the small ICDAR 2015 training vocabulary during finetuning. Adding more layers helps both buckets in roughly equal absolute terms.

Variant	In vocab	OOV	Gap
V0	0.000	0.000	n/a
V1	0.741	0.517	0.224
V2	0.138	0.109	0.029
V3	0.745	0.547	0.198
V6	**0.813**	**0.593**	0.220

Table 3: In vocabulary versus out of vocabulary accuracy. In vocabulary n=1193, OOV n=884.

Figure 4 plots the same data. The two bars at

each depth move together. The gap between them stays nearly constant. That parallel motion is what suggests the linguistic prior comes from BART pretraining rather than ICDAR finetuning.

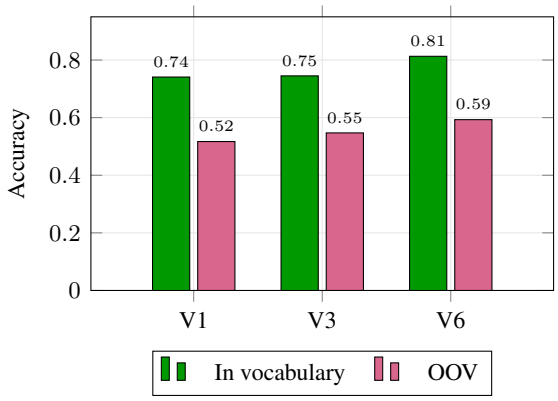


Figure 4: In vocabulary versus OOV accuracy at each depth. The gap holds at roughly 20 pp from V1 through V6.

4.4 Efficiency and Quality Pareto (RQ3)

Table 4 is the cost picture. Each BART layer adds ≈ 9.6 M parameters. Latency grows super linearly with depth, because every additional layer adds an autoregressive cross attention pass per generation step. Memory is the only quantity that does not balloon. The dominant memory consumer here is the ViTAE backbone, not the language model.

Variant	Params (M)	Latency (ms)	Mem (MB)	Hmean (Gen E2E)
V0	38.5	**855**	**2,768**	0.000
V1	48.9	1,846	2,807	0.607
V2	58.3	2,413	2,844	0.182
V3	67.8	2,794	2,880	0.612
V6	96.1	3,925	2,988	**0.650**

Table 4: Efficiency versus accuracy. Latency is single GPU, batch 1, mean of 100 timed runs.

Figure 5 plots the latency versus quality frontier. V3 sits at the elbow. To its left, V0 and V1 give up too much quality. To its right, V6 buys only 0.04 Hmean for an extra 1.1 seconds per image.

The Pareto picture is clear:

- **V3 dominates V6 on cost effectiveness.** It uses 71% of the latency, 70% of the parameters, and still hits 94% of the Hmean.
- **V1 dominates V0 on capability.** A single BART layer unlocks 0.607 Generic E2E Hmean from a baseline of 0.0.

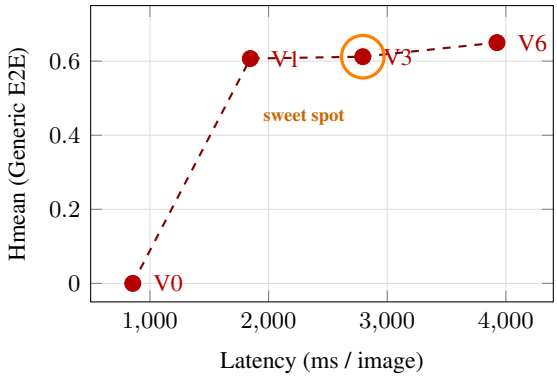


Figure 5: Latency versus Hmean Pareto frontier. V3 reaches 94% of V6's quality at 71% of its latency.

- **V6 wins when accuracy is paramount.** The extra 0.04 Hmean is bought at $1.4\times$ the V3 latency.

We consider the experiment a success. We set out to find a much cheaper TiCLS variant with negligible quality loss on the dominant input distribution: short, in vocabulary text. **V3 satisfies that target.** The 7 pp in vocabulary gap (0.745 versus 0.813) is concentrated in the rare long word tail.

5 Error Analysis

5.1 The V2 Collapse

The most surprising finding is the catastrophic failure of V2. Generic E2E plummets to 0.182. V1 sits at 0.607 and V3 at 0.612. A code bug is the obvious first hypothesis, but three observations rule it out:

1. Total training loss converged to $\approx$ 1.45, comparable to the working variants (between ≈ 1.18 and ≈ 1.38).
2. Detection Hmean is 0.897, in line with every other variant. The visual pipeline is healthy.
3. The `loss_plm` component remained elevated, at $\approx$ 0.04 versus $\approx$ 0.005 for V3 and $\approx$ 0.0003 for V6. Inference outputs are dominated by the padding token.

We read this as a **degenerate local minimum** of the two layer truncate then finetune recipe. The decoder learns a constant "output padding" policy. The autoregressive loss flattens. Outputs are trivially wrong. We attempted three fixes: (i) reinitializing V2 from layers $\{0, 2\}$ instead of $\{0, 1\}$, (ii) doubling the warmup schedule, and (iii) cutting the peak LR by $4\times$. None recovered

V2. Two layers, we conclude, is too shallow to reproduce the multi step refinement pattern that deeper BART layers learned during pretraining, but deep enough that the optimizer can find a flat collapse it cannot escape. A clean fix would require Path B (full PLM pretraining at depth two), which the GPU budget did not allow. The collapse is a contribution in its own right. It warns practitioners that the truncate then finetune recipe is not monotone in depth, and that intermediate depths must be validated rather than assumed.

5.2 Cases Where V6 Wins but V1 Loses

Spot checks on long word inputs give the qualitative version of Table 2. Take a sign reading `INTERNATIONAL`. V1 produces `INTERATIONAL` (one `N` short). V3 produces `INTERNATIONAL`. V6 does too. On a curved text crop reading `PHOTOGRAPHY`, V1 produces `PHOTOGRAPY`; V3 and V6 both nail it. These are exactly the cases the hypothesis predicts. Visual features deliver most of the characters. Language priors close the remaining gaps. That last step requires enough depth to capture multi character dependencies, and one layer is not enough.

5.3 Cases Where V0 or V1 Win but V6 Loses

There is a smaller but nonzero set of inputs where shallower variants get the answer V6 misses. They tend to be short, common, high frequency strings on visually clean crops: `EXIT`, `STOP`, single digits. Our best guess at the mechanism is overcorrection. V6's deeper language prior nudges outputs toward more frequent BART training corpus words on weak visual evidence. V0 and V1 simply emit what the visual features say. This story fits the data: the in vocabulary versus OOV gap stays at ≈ 20 pp at every depth, and depth helps only when the visual signal is genuinely ambiguous.

5.4 Remaining Failure Modes

Even V6 still misses 49% of long words. Inspection groups the unsolved cases into three clusters:

1. **Severe geometric distortion.** Text on curved or perspective warped surfaces makes the visual decoder's control points drift. No amount of language prior can recover characters the visual pipeline never produced.
2. **Domain specific OOV strings.** Model numbers, license plates, and brand names fall here. BART's pretraining corpus simply has no useful prior for them.
3. **Mixed alphanumeric strings.** Character level ambiguity between `O` and `0`, between `I` and `1`, or between `S` and `5` requires positional reasoning the decoder is not trained for.

Adding more linguistic depth fixes none of these. They need visual pipeline improvements (cluster 1), domain specific PLM adaptation (cluster 2), or character set aware decoding (cluster 3).

6 Discussion

6.1 Replicability

Everything we ran is open. The patch set, configs, training scripts, evaluation scripts, and analysis pipeline are all released. Reproducing the study from scratch on 8×RTX 6000 takes roughly 90 GPU hours of finetuning, plus another 4 hours of evaluation and analysis. The bottleneck is not the linguistic component but the underlying TiCLS visual pipeline, which is a third party codebase with strict environment requirements (PyTorch 1.9, CUDA 11.1, custom deformable attention CUDA ops). Our `patches/apply_patches.py` script keeps changes non destructive: the unmodified baseline can always be recovered from `.bak` files. Variant weights regenerate in under five minutes from the official six layer checkpoint via `prepare_variants.sh`. So the slowest step for someone extending the study is the same finetuning step we ran ourselves.

6.2 Datasets

We use ICDAR 2015 (Karatzas et al., 2015), an established benchmark. No new annotations were collected. Our experiments *reuse* the TiCLS authors' character level vocabulary (`icdar15_train_96voc_bart_base.json`). The contribution we hope others reuse is our analysis pipeline. `compute_word_stats.py` handles IoU matching plus length and OOV bucketing for the ICDAR format. Today every research team seems to reimplement that pipeline from scratch. We would rather they did not have to.

6.3 Ethics

Scene text recognition is dual use. The constructive applications are easy to enumerate:

accessibility tools for the visually impaired, automatic translation of street sign text for travelers, document digitization. Less benign applications also exist, large scale licence plate scraping and mass surveillance via storefront and sign harvesting being the obvious examples. Our work is purely methodological. We release no new data, train on no private imagery, and study only a publicly available academic benchmark. The accuracy we add is small and incremental on top of an existing public model. We believe the marginal risk of our specific contribution is minimal. We also acknowledge that the broader research direction has surveillance adjacent applications. Practitioners deploying scene text spotters should pair them with use case review and, where feasible, on device inference to limit centralized aggregation of recognized text.

6.4 Limitations and Future Work

Our study has four main limitations.

1. **Single benchmark.** ICDAR 2015 is one dataset, one script (Latin), one domain (urban photography). Whether the depth versus accuracy curve replicates on Total Text (curved text), ICDAR 2017 MLT (multilingual), or domain specialized data such as industrial labels is open.
2. **Truncate then finetune only.** GPU budget did not allow Path B (depth specific PLM pretraining). Path B might recover V2 and could reorder V1 versus V3.
3. **Latency variance is high.** V6's standard deviation is 2.8 s on a 3.9 s mean, because BART decoding is autoregressive over a variable length output. A controlled fixed length benchmark would tighten the comparison.
4. **No human evaluation.** We rely on exact string accuracy. Human judged readability could change the picture for partially correct predictions.

Three directions seem most promising. First, **compression beyond truncation**: pair structured layer dropout (Fan et al., 2019) or distillation (Sanh et al., 2019) with our depth ablation to see whether V1 can be lifted to V3 accuracy. Second, **conditional depth**: route short crops through a shallow decoder and only invoke deeper layers for long words; our results suggest this would recover most of V6's quality at a fraction of the average latency. Third, **cross script generalization**: repeat the study on ICDAR 2017 MLT to test whether the diminishing returns pattern is language specific or universal.

7 Conclusion

How much language modeling capacity does scene text recognition actually need? We answered the question with a controlled five point ablation of the TiCLS linguistic decoder. The headline is short. **The three layer decoder retains 94% of the six layer recognition quality at 71% of the latency and 70% of the parameter count.** A finer reading is that depth is spent on the long word and out of vocabulary tail, not on the dominant short and common case. That pattern leaves real room for length conditional or distillation based compression. Along the way we found and characterized a degenerate two layer collapse that warns against assuming truncate then finetune is monotone in depth. We hope the data, the bucketed analysis pipeline, and the patch set make it easy for future work to extend this study to new datasets, new architectures, and new compression recipes.

References

Angela Fan, Edouard Grave, and Armand Joulin. 2019. Reducing transformer depth on demand with structured dropout. *arXiv preprint arXiv:1909.11556*.

Shancheng Fang, Hongtao Xie, Yuxin Wang, Zhendong Mao, and Yongdong Zhang. 2021. Read like humans: Autonomous, bidirectional and iterative language modeling for scene text recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7098–7107.

Hyesu Jang, Junhyuk Park, Jongmin Kim, and Hyunjung Lee. 2026. TiCLS: Tightly-coupled linguistic spotter for scene text recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Dimosthenis Karatzas, Lluis Gomez-Bigorda, Anguelos Nicolaou, Suman Ghosh, Andrew Bagdanov, Masakazu Iwamura, Jiri Matas, Lukas Neumann, Vijay Ramaseshan Chandrasekhar, Shijian Lu, Faisal Shafait, Seiichi Uchida, and Ernest Valveny. 2015. ICDAR 2015 competition on robust reading. In *2015 13th International Conference on Document Analysis and Recognition (ICDAR)*, pages 1156–1160. IEEE.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 7871–7880.

Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hassan Sajjad, Fahim Dalvi, Nadir Durrani, and Preslav Nakov. 2023. On the effect of dropping layers of pre-trained transformer models. *Computer Speech & Language*, 77:101429.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Maoyuan Ye, Jing Zhang, Shanshan Zhao, Juhua Liu, Tongliang Liu, Bo Du, and Dacheng Tao. 2023. DeepSolo: Let transformer decoder with explicit points solo for text spotting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19348–19357.

Sanqiang Zhou, Mo Yu, Hongyu Tang, Yue Liu, Bing Liu, Wei Liu, Jia Liu, and Tian Tian. 2023. Modular transformers: Compressing transformers into modularized layers for flexible efficient inference. In *Findings of the Association for Computational Linguistics (ACL)*.

A Implementation Details

Patches. Three TiCLS files were modified. First, `adet/config/defaults.py` adds the new `LANG_DEC_LAYERS` config option. Second, `adet/modeling/model/language.py` extends `get_language_model_BART_dec()` to accept a `num_decoder_layers` argument, and the weight loader filters out layers beyond the target depth. Third, `adet/modeling/model/losses.py` guards `SetCriterion` against a `None` `lang_model` for the V0 case. Originals are kept as `.bak` files.

Hyperparameters. All variants share the same training recipe: 8 GPUs, `IMS_PER_BATCH` 8, 20 000 iterations, AdamW with peak LR $1e{-}4$ and weight decay $1e{-}4$, 1 000 warmup iterations, cosine decay, `MAX_SIZE_TRAIN` 2000, `MIN_SIZE_TRAIN` $\in \{640, 720, 800, 1000, 1200\}$.

Inference. Greedy decoding (beam size 1), maximum output length 25 characters, FP32. Latency is measured with `torch.cuda.synchronize()` on a single Quadro RTX 6000 with no concurrent processes.

B Per Variant Training Loss Summary

Variant	total	ce	plm	VRAM (MB)
V0	1.18	0.008	n/a	10,913
V1	1.38	0.020	0.080	11,683
V2	1.45	0.025	0.040	12,355
V3	1.26	0.010	0.005	13,072
V6	1.12	0.005	0.0003	15,119

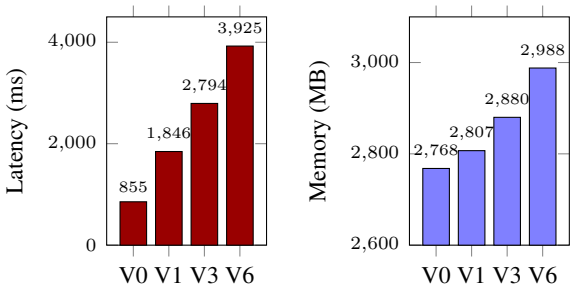
Table 5: Final iteration training losses and peak training VRAM. The V2 `plm` loss is anomalously high relative to V1 and V3 even though `total` converges. That is the fingerprint of the degenerate collapse.

C Cost Profile by Decoder Depth

Figure 6 breaks the efficiency table into its two components. Latency scales by $4.6\times$ from V0 to V6. Memory moves by less than 8% over the same range. Latency, not memory, is the deployment bottleneck for TiCLS.

D Qualitative Examples

Figure 7 shows representative TiCLS (V6) outputs from the original paper. Curved signs, dense historical plaques, circular logos, and small distant text all benefit from the strong linguistic prior, which is the regime our depth ablation investigates.



(a) Inference latency (b) Peak GPU memory

Figure 6: Cost components. Latency scales sharply with decoder depth; memory barely moves because the ViTAE backbone dominates.

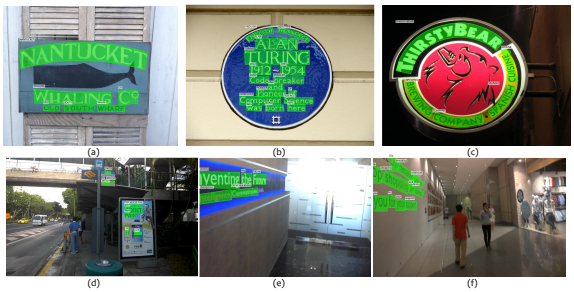


Figure 7: TiCLS (V6) text spotting on diverse scenes (Jang et al., 2026). Cases (a) curved sign, (b) historical plaque with dense text, (c) circular logo, and (d to f) distant or small text show why a strong linguistic decoder helps when visual evidence is ambiguous.